```
# Add these imports to the top of app.py
from duckduckgo_search import DDGS
import requests
from bs4 import BeautifulSoup
from datetime import datetime
import json

# Enhanced search with multiple categories
def advanced_search(query, search_type="text"):
    """
    Advanced search with different types

    Args:
        query: Search query string
        search_type: "text", "news", or "images"
    """
    try:
        with DDGS() as ddgs:
            if search_type == "news":
                # Search news specifically
                results = list(ddgs.news(query,
max_results=5))
                summary = "📰 Latest News:\n\n"
                for i, result in enumerate(results,
1):
                    date = result.get('date',
'Recent')
                    summary += f"{i}.
{result['title']}\n"
                    summary += f"   Source:
{result.get('source', 'Unknown')}\n"
                    summary += f"   Date: {date}\n"
                    summary += f"   {result['body']
[:200]}...\n\n"

            elif search_type == "images":
                # Search images (returns URLs)
                results = list(ddgs.images(query,
```

```
max_results=5))
                summary = "🖼️ Image Results:\n\n"
                for i, result in enumerate(results,
1):
                    summary += f"{i}.
{result['title']}\n"
                    summary += f"   URL:
{result['image']}\n"
                    summary += f"   Source:
{result.get('url', '')}\n\n"

            else:
                # Regular text search with time limit
                results = list(ddgs.text(
                    query,
                    max_results=5,
                    timelimit='m'  # Last month
                ))
                summary = "🔍 Search Results:\n\n"
                for i, result in enumerate(results,
1):
                    summary += f"{i}.
{result['title']}\n"
                    summary += f"   {result['body']
[:250]}...\n"
                    summary += f"   🔗
{result['href']}\n\n"

            return summary if results else "No results
found."

    except Exception as e:
        return f"Search error: {str(e)}"

# Fetch and summarize web pages
def fetch_webpage_content(url, max_chars=2000):
    """
    Fetch actual content from a webpage

    Args:
        url: Webpage URL
        max_chars: Maximum characters to extract
```

```
    """
    try:
        headers = {
            'User-Agent': 'Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36'
        }
        response = requests.get(url, headers=headers,
timeout=10)
        response.raise_for_status()

        soup = BeautifulSoup(response.text,
'html.parser')

        # Remove script and style elements
        for script in soup(["script", "style", "nav",
"footer", "header"]):
            script.decompose()

        # Get text
        text = soup.get_text(separator='\n',
strip=True)

        # Clean up extra whitespace
        lines = [line.strip() for line in
text.splitlines() if line.strip()]
        text = '\n'.join(lines)

        # Limit length
        if len(text) > max_chars:
            text = text[:max_chars] + "..."

        return text

    except Exception as e:
        return f"Could not fetch webpage: {str(e)}"

# Fact-checking helper
def fact_check_query(claim):
    """
    Search for fact-checks on a claim
    """
    try:
```

```
        with DDGS() as ddgs:
            # Search fact-checking sites
            query = f"{claim} site:snopes.com OR
site:factcheck.org OR site:politifact.com"
            results = list(ddgs.text(query,
max_results=3))

            if results:
                summary = "🔍 Fact Check Results:\n\n"
                for i, result in enumerate(results,
1):
                    summary += f"{i}.
{result['title']}\n"
                    summary += f"   {result['body']
[:200]}...\n"
                    summary += f"   🔗
{result['href']}\n\n"
                return summary
            else:
                return "No fact-check information
found. Claim could not be verified."

    except Exception as e:
        return f"Fact-check search error: {str(e)}"

# Wikipedia quick lookup
def wiki_search(query):
    """
    Quick Wikipedia summary
    """
    try:
        with DDGS() as ddgs:
            query_wiki = f"{query} site:wikipedia.org"
            results = list(ddgs.text(query_wiki,
max_results=1))

            if results:
                return f"📚 Wikipedia: {results[0]
['body']}"
            return "No Wikipedia article found."

    except Exception as e:
```

```
        return f"Wikipedia search error: {str(e)}"

# Add to your Flask routes
@app.route('/research', methods=['POST'])
def research():
    """Advanced research endpoint"""
    data = request.json
    query = data.get('query', '')
    research_type = data.get('type', 'text')  # text,
news, fact_check, wiki

    if not query:
        return jsonify({"error": "No query
provided"}), 400

    try:
        if research_type == 'news':
            result = advanced_search(query, 'news')
        elif research_type == 'fact_check':
            result = fact_check_query(query)
        elif research_type == 'wiki':
            result = wiki_search(query)
        else:
            result = advanced_search(query, 'text')

        return jsonify({"results": result})

    except Exception as e:
        return jsonify({"error": str(e)}), 500

# Modified generate_response with enhanced search
def generate_response_enhanced(user_message,
conversation_id, use_search=False):
    """Enhanced version with better search
integration"""

    if conversation_id not in conversations:
        conversations[conversation_id] = []

    history = conversations[conversation_id]
    search_context = ""
```

```
    # Smart search detection
    if use_search:
        # Detect what kind of search needed
        lower_msg = user_message.lower()

        if any(word in lower_msg for word in ['news',
'latest', 'recent']):
            search_context =
f"\n\n{advanced_search(user_message, 'news')}\n"
        elif any(word in lower_msg for word in
['fact', 'true', 'false', 'verify']):
            search_context =
f"\n\n{fact_check_query(user_message)}\n"
        elif 'wikipedia' in lower_msg or 'wiki' in
lower_msg:
            search_context =
f"\n\n{wiki_search(user_message)}\n"
        else:
            search_context =
f"\n\n{advanced_search(user_message, 'text')}\n"

    # Rest of the function same as before...
    prompt = f"<|im_start|>system\n{SYSTEM_PROMPT}
<|im_end|>\n"

    for msg in history[-10:]:
        role = msg['role']
        content = msg['content']
        prompt += f"<|im_start|>{role}\n{content}
<|im_end|>\n"

    if search_context:
        prompt += f"
<|im_start|>user\n{user_message}\n\nContext from web
search:{search_context}<|im_end|>\n"
    else:
        prompt += f"<|im_start|>user\n{user_message}
<|im_end|>\n"

    prompt += "<|im_start|>assistant\n"

    output = llm(
```

```
        prompt,
        max_tokens=512,
        temperature=0.8,
        top_p=0.95,
        repeat_penalty=1.1,
        stop=["<|im_end|>", "User:", "\nUser:",
"\n\nUser:"]
    )

    response_text = output['choices'][0]
['text'].strip()

    history.append({"role": "user", "content":
user_message})
    history.append({"role": "assistant", "content":
response_text})

    if len(history) > 20:
        conversations[conversation_id] = history[-20:]

    return response_text

# Add to requirements.txt:
# beautifulsoup4==4.12.2
# requests==2.31.0

"""
USAGE IN YOUR INTERFACE:

Add these buttons in index.html:

<div class="research-buttons">
    <button onclick="researchNews()">📰 News</button>
    <button onclick="researchWiki()">📚
Wikipedia</button>
    <button onclick="factCheck()">🔍 Fact
Check</button>
</div>

<script>
async function researchNews() {
    const query = prompt("What news would you like to
```

```
search for?");
    if (!query) return;

    const response = await fetch('/research', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({query: query, type:
'news'})
    });

    const data = await response.json();
    addMessage(data.results, 'assistant');
}

async function researchWiki() {
    const query = prompt("What would you like to learn
about?");
    if (!query) return;

    const response = await fetch('/research', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({query: query, type:
'wiki'})
    });

    const data = await response.json();
    addMessage(data.results, 'assistant');
}

async function factCheck() {
    const claim = prompt("What claim would you like to
fact-check?");
    if (!claim) return;

    const response = await fetch('/research', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({query: claim, type:
'fact_check'})
    });
```

```
    const data = await response.json();
    addMessage(data.results, 'assistant');
}
</script>
"""
```